Coordinate frames for the Curve25519-family factor base (2026-11-03)

Class: a mechanism measurement with exact counts and a component result on the algebraic track. It is not a leaderboard table A–C row, and it does not change the order against the generic baseline or rho. Protocol: `PROTOCOL.md`, frozen in `6a800b1513` before any confirmation count. Source of this report: `README.md` (source of truth) and the predictions of `PROTOCOL.md`.

Question (owner)

Do coordinate systems other than the current one (other Montgomery forms, projective, Edwards, …) yield more relations for Curve25519 index calculus?

Answer

1. **Which model you use does not matter. Which frame you pick does.**
   - Every degree-2 coordinate on E is a Möbius frame of x. Weierstrass x, Montgomery u, Edwards y and the Jacobi quartic coordinate are all frames of this kind. Projective coordinates change arithmetic, not the factor base.
   - Hessian, twisted Hessian, Jacobi intersection and Huff models are not defined over F_p for this family, since #E = 8r with cyclic Z/8.
   - Over 2,000 + 2,000 random frames per (curve, t), Weierstrass `bits lo=0` and Edwards y are typical draws. All 50/50 cells are inside the central 98% (P1).
   - Translation and Möbius frames have the same distribution (P2).
   - Edwards "beat" Weierstrass in `curve25519_edwards_semaev_20261102` only because `lo = 0` was a below-average draw on that curve.
2. **On the generic track (the baseline) there is no lever.** For any N-column base, P(target = ±P_i ± P_j) ≤ #signed pairs / r. The projected base already sits at that ceiling (`curve25519_algebraic_ladder_20261102`, Q-A).
3. **On the algebraic track (`pkm-semaev`), choosing the frame is a real gain.**
   - The F4 system has the same degree and size in every frame.
   - Exact coverage depends on how many points the frame holds and on how their cofactor cosets pair up, and both fluctuate from frame to frame.
   - Choosing the frame per (curve, t) without discrete logs (argmax over coset-cancelling pairs, read from [r]P) raises coverage at an unchanged per-target F4 cost. In the engine, at k = 20, the F4 mul-adds per relation fall **4.71× (t = 5)** and **3.33× (t = 6)** (P5).

Results

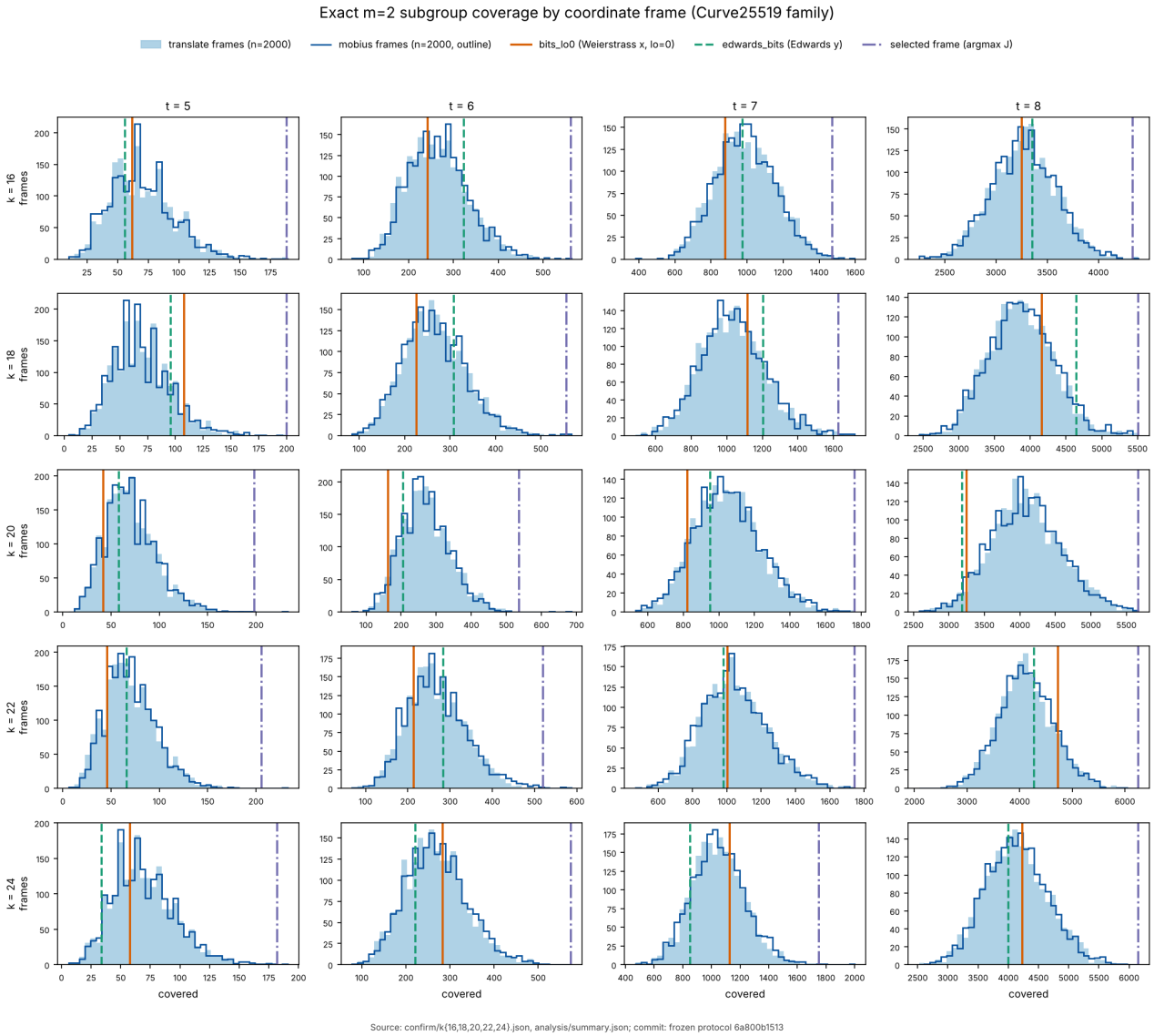


Figure 1: Coverage distributions (`figures/coverage_distributions.png`).

Census (exact, five family miniatures; k = 16, 18, 22, 24 held out)

Selected frame ÷ `bits lo=0` (and ÷ the mean frame):

k	t = 4	t = 5	t = 6	t = 7	t = 8
16	12.00 (3.75)	3.03 (2.66)	2.32 (2.13)	1.67 (1.49)	1.33 (1.32)
18	3.82 (4.46)	1.85 (2.85)	2.46 (2.09)	**1.46** (1.58)	1.32 (1.43)
20	5.33 (3.38)	4.71 (2.84)	3.31 (1.98)	2.14 (1.68)	1.75 (1.38)
22	4.57 (3.31)	4.48 (2.94)	2.43 (1.95)	1.74 (1.67)	1.32 (1.52)
24	4.30 (4.45)	3.14 (2.62)	2.02 (2.16)	1.56 (1.68)	1.45 (1.49)

Source: `analysis/summary.json` (`scripts/analyse.py` over `confirm/k*.json`). The confirm data files are in the artifact store, with the sha256 values in `confirm/SHA256SUMS` and `confirm/.artifacts.json`.

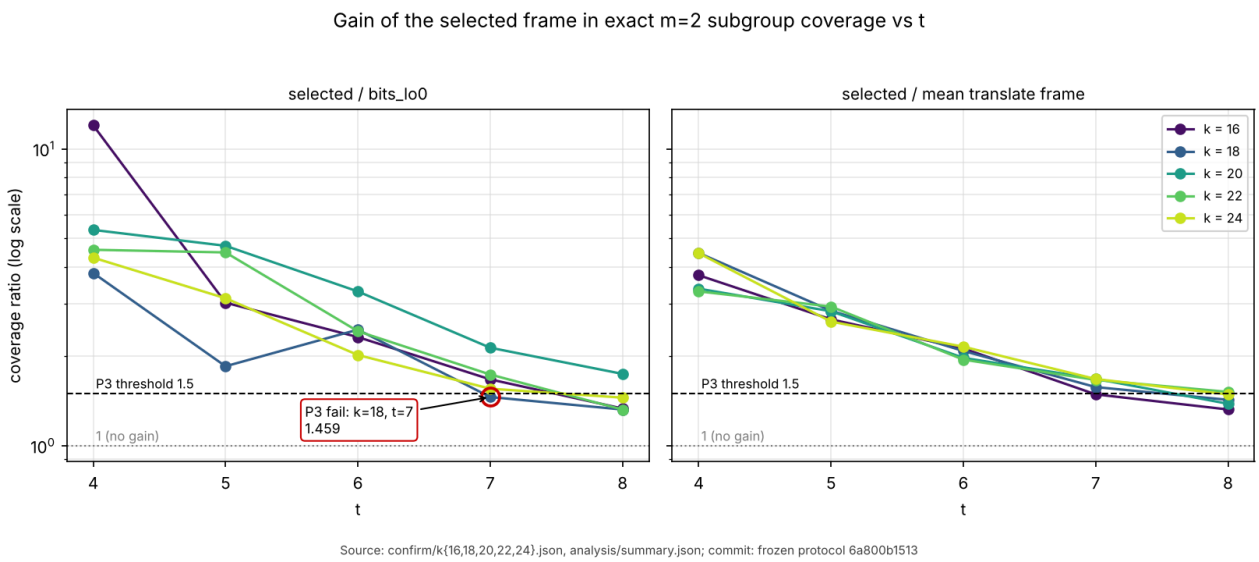


Figure 2: Gain versus t (`figures/gain_vs_t.png`).

The gain over the mean frame barely depends on the curve (right panel above). It falls with t, as the extreme-value model predicts: the fluctuation is about $2^{t/2}$ against a mean of 2^{t-1}.

Engine (P5): `curve25519_family f4cost 20 …`, 100 targets, seed 2026111500+t

The binary is `0459a725…b4c5` (`engine/binary.sha256`), built from this branch.

cell	cols	covered	max degree	terms	mul-adds/target	F4 mul-adds / relation
`bits t=5 G=3`	11	42	5	64	3,036,911.76	$9.487 \cdot 10^{9}$
`bits@976503 t=5 G=3`	24	198	5	64	3,036,916.11	**$2.012 \cdot 10^{9}$** (0.212×)
`bits t=6 G=3`	25	162	6	154	172,425,384.63	$1.396 \cdot 10^{11}$
`bits@908172 t=6 G=3`	42	536	6	154	171,133,755.53	**$4.189 \cdot 10^{10}$** (0.300×)
`bits t=5 G=0` (extra)	11	42	6	256	127,284,344.31	$3.976 \cdot 10^{11}$
`bits@976503 t=5 G=0` (extra)	24	198	6	256	127,284,307.74	**$8.434 \cdot 10^{10}$** (0.212×)

- `coverage_exact` equals the census exactly.
- The `bits` arm reproduces the published cells of `curve25519_edwards_semaev_20261102` exactly.
- One relation was found and re-summed in the frame t = 6 cell.

Predictions

Statements as frozen in `PROTOCOL.md` (each decided by exact counts):

- **P1, no model effect.** `bits_lo0` and `edwards_bits` lie inside the central 98% of the random translation-frame coverage distribution in at least 45 of the 50 (curve, t, frame) cells.
- **P2, translation ≡ Möbius.** Mean coverage translation ÷ Möbius is in [0.93, 1.07] in every (curve, t) cell with mean covered ≥ 50.
- **P3, frame gain.** The selected frame's covered ÷ `bits_lo0` covered is ≥ 1.5 at t ∈ {5, 6, 7} on every held-out curve. The ratios at t = 4 and 8 are reported, with no threshold.
- **P4, log-free selection suffices.** The selected (argmax J) frame's coverage is ≥ 0.9 × the oracle-best (argmax covered) translation frame's, in every cell.
- **P5, the engine agrees** (k = 20, t ∈ {5, 6}, G = 3, 100 targets, frame selected in the k = 20 replication). For `curve25519_family f4cost 20 bits@L* t 3 …` against `bits t 3 …` with the same seed: `coverage_exact` equals the census exactly; `solving_degree_max` and `poly_terms_per_specialisation` are equal; mul-adds/target are within ±5%. So F4 mul-adds per relation (at exact coverage) fall by the coverage ratio.

Outcomes:

prediction	outcome
P1 no model effect	**held**, 50/50 cells inside 1–99%

P2 translation ≡ Möbius	**held**, 0.986–1.018
P3 ≥ 1.5× vs `bits lo=0` at t = 5–7, held-out curves	**failed** in 1 of 12 cells: k = 18, t = 7 gives 1.459×. There `lo = 0` was itself a lucky draw (68th percentile); against the mean frame the gain is 1.58×. The other 11 cells give 1.56–4.48×.
P4 log-free selector suffices	**held**: equal to the oracle-best frame in 23/25 cells, ≥ 0.99 in all
P5 engine agrees	**held**: same coverage, degree and terms; mul-adds/target ratio 1.000, 0.993, 1.000

Failure handling (PROTOCOL.md). A failed prediction is reported as failed and not retuned. If P5 fails on polynomial size, the gain is not a free gain and the result is reported as a trade-off.

Costs and scope (distance remaining)

- **Frame search is priced.** K = 2000 frames cost 2000·2^t Legendre symbols plus [r]P on every column found, about 1.5·log2 r group operations each. At k = 20, t = 6 that is about $1.6 \cdot 10^6$ group operations. This is small against one relation's F4 work ($4 \cdot 10^{10}$ mul-adds), but large against a whole rho solve at k = 20, which takes about 10^3 group operations. On the algebraic track it is noise. It is not free against rho.
- **The algebraic track is still far behind.** The best algebraic arm was 2.6× (w = 0) to $1.4 \cdot 10^5$× (w = 0.1) behind the generic baseline (`curve25519_algebraic_ecbench_20261102`). The generic baseline itself is 7.8× rho at k = 20. A 3–5× cut in F4 work per relation does not change any order across the §5a band. The distance remaining to rho on this track is still about 10^4–10^5× at w ≥ 0.1.
- **The gain shrinks with t**: 1.3–1.5× against the mean frame at t = 8. Nothing is extrapolated to 255 bits.
- **Not explained:** the single-seed Edwards t = 5, G = 0 per-target ratio of 0.646 from the earlier study. A translation frame at the same cell costs exactly the same as `bits` (127,284,307 against 127,284,344), so that ratio is not a generic frame effect.

Claim scope (PROTOCOL.md). Algebraic track only. That track is far behind the generic baseline, 2.6× at w = 0 and $1.4 \cdot 10^5$× at w = 0.1. A coverage gain of a few × does not change the order between them across the §5a band, and rho remains ahead. The distance remaining is reported, not a pass/fail. This is not a leaderboard table A–C row; at most it is a component result of the algebraic track. The miniatures are k ≤ 24. The relative gain shrinks as t grows: extreme-value fluctuations scale as $2^{t/2}$ against a mean of 2^{t-1}. No 255-bit extrapolation is claimed.

Next

- Use the frame selector as the default for `pkm-semaev` bits bases, and re-price `curve25519_algebraic_ecbench` with it.
- Search frames jointly with the guess split G, and on the PKM isogeny tower (the best algebraic arm), where the same coset argument applies.

Reproduce

```
scripts/run_confirm.sh          # census, ~1 min (python3 + numpy)
python3 scripts/analyse.py      # P1–P4
cargo build --release --bin curve25519_family
BIN=target/release/curve25519_family scripts/run_engine.sh   # P5, ~1 min
python3 scripts/figures.py      # needs matplotlib
```

Deviations

- Conductor and the AWS CLI were absent in this container. There was no Conductor task, no S3 resume push and no EKS route.
- Every measurement is a deterministic exact count (census) or a counted F4 mul-add, never a timing, and they were run locally.